

"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Linked Data Structures

Advanced Programming

FCSE 2026

Introduction to Linked Data Structures

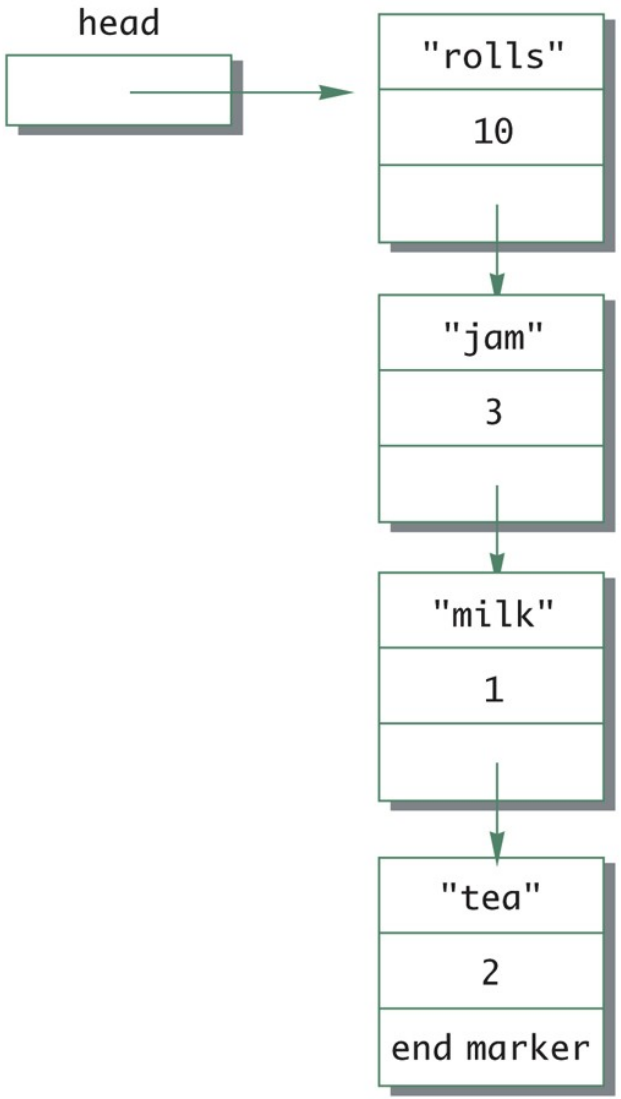
- A *linked data structure* consists of capsules of data known as *nodes* that are connected via *links*
  - Links can be viewed as arrows and thought of as one way passages from one node to another
- In Java, nodes are realized as objects of a node class
- The data in a node is stored via instance variables
- The links are realized as references
  - A reference is a memory address, and is stored in a variable of a class type
  - Therefore, a link is an instance variable of the node class type itself

Java Linked Lists

- The simplest kind of linked data structure is a *linked list*
- A linked list consists of a single chain of nodes, each connected to the next by a link
  - The first node is called the *head* node
  - The last node serves as a kind of end marker

Nodes and Links in a Linked List

Display 15.1 Nodes and Links in a Linked List



A Simple Linked List Class

- In a linked list, each node is an object of a node class
  - Note that each node is typically illustrated as a box containing one or more pieces of data
- Each node contains data and a link to another node
  - A piece of data is stored as an instance variable of the node
  - Data is represented as information contained within the node "box"
  - Links are implemented as references to a node stored in an instance variable of the node type
  - Links are typically illustrated as arrows that point to the node to which they "link"

A Node Class (Part 1 of 3)

Display 15.2 A Node Class

```
public class Node1
{
    private String item;
    private int count;
    private Node1 link;

    public Node1()
    {
        link = null;
        item = null;
        count = 0;
    }

    public Node1(String newItem, int newCount, Node1 linkValue)
    {
        setData(newItem, newCount);
        link = linkValue;
    }
```

A node contains a reference to another node. That reference is the link to the next node.

We will define a number of node classes so we numbered the names, as in Node1.

(continued)

A Node Class (Part 2 of 3)

Display 15.2 A Node Class

```
public void setData(String newItem, int newCount)
{
    item = newItem;
    count = newCount;
}

public void setLink(Node1 newLink)
{
    link = newLink;
}
```

We will give a better definition of a node class later in this chapter.

(continued)

A Node Class (Part 3 of 3)

Display 15.2 A Node Class

```
    public String getItem()
    {
        return item;
    }

    public int getCount()
    {
        return count;
    }

    public Node1 getLink()
    {
        return link;
    }
}
```

A Simple Linked List Class

- The first node, or start node in a linked list is called the head node
  - The entire linked list can be traversed by starting at the head node and visiting each node exactly once
- There is typically a variable of the node type (e.g., **`head`**) that contains a reference to the first node in the linked list
  - However, it is not the head node, nor is it even a node
  - It simply contains a reference to the head node

A Simple Linked List Class

- A linked list object contains the variable **`head`** as an instance variable of the class
- A linked list object does not contain all the nodes in the linked list directly
  - Rather, it uses the instance variable **`head`** to locate the head node of the list
  - The head node and every node of the list contain a link instance variable that provides a reference to the next node in the list
  - Therefore, once the head node can be reached, then every other node in the list can be reached

An Empty List Is Indicated by `null`

- The **`head`** instance variable contains a reference to the first node in the linked list
  - If the list is empty, this instance variable is set to **`null`**
  - Note: This is tested using **`==`**, not the **`equals`** method
- The linked list constructor sets the head instance variable to **`null`**
  - This indicates that the newly created linked list is empty

A Linked List Class (Part 1 of 6)

Display 15.3 A Linked List Class

```
1   public class LinkedList1
2   {
3       private Node1 head;
4
5       public LinkedList1()
6       {
7           head = null;
8       }

9       /**
10       Adds a node at the start of the list with the specified data.
11       The added node will be the first node in the list.
12      */
13      public void addToStart(String itemName, int itemCount)
14      {
15          head = new Node1(itemName, itemCount, head);
16      }
```

We will define a better linked list class later in this chapter.

(continued)

A Linked List Class (Part 2 of 6)

Display 15.3 A Linked List Class

```
17      /**
18       Removes the head node and returns true if the list contained at least
19       one node. Returns false if the list was empty.
20      */
21      public boolean deleteHeadNode()
22      {
23          if (head != null)
24          {
25              head = head.getLink();
26              return true;
27          }
28          else
29              return false;
30      }
```

(continued)

A Linked List Class (Part 3 of 6)

Display 15.3 A Linked List Class

```
31      /**
32       Returns the number of nodes in the list.
33      */
34      public int size()
35      {
36          int count = 0;
37          Node1 position = head;
38
```

(continued)

A Linked List Class (Part 4 of 6)

Display 15.3 A Linked List Class

```
39          while (position != null)
40          {
41              count++;
42              position = position.getLink();
43          }
44          return count;
45      }

46      public boolean contains(String item)
47      {
48          return (find(item) != null);
49      }
```

The last node is indicated by the `link` *field being equal to* `null`.

(continued)

A Linked List Class (Part 5 of 6)

Display 15.3 A Linked List Class

```
50      /**
51       Finds the first node containing the target item, and returns a
52       reference to that node. If target is not in the list, null is returned.
53      */
54      private Node1 find(String target)
55      {
56          Node1 position = head;
57          String itemAtPosition;
58          while (position != null)
59          {
```

(continued)

A Linked List Class (Part 6 of 6)

Display 15.3 A Linked List Class

```
60              itemAtPosition = position.getItem();
61              if (itemAtPosition.equals(target))
62                  return position;
63              position = position.getLink();
64          }
65          return null; //target was not found
66      }

67      public void outputList()
68      {
69          Node1 position = head;
70          while (position != null)
71          {
72              System.out.println(position.getItem() + " "
73                                                 + position.getCount());
74              position = position.getLink();
75          }
76      }
77  }
```

This is the way you traverse an entire linked list.

Indicating the End of a Linked List

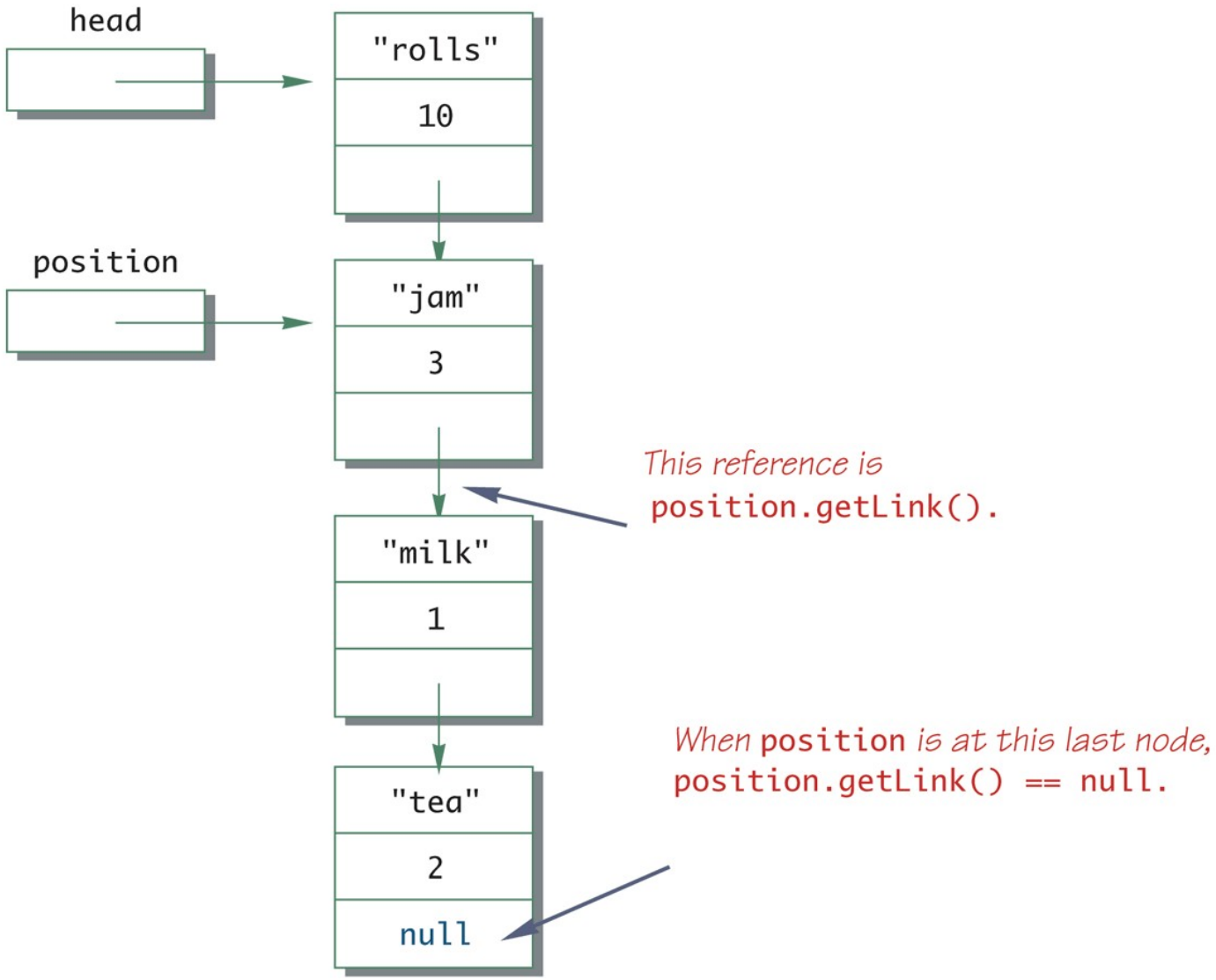
- The last node in a linked list should have its link instance variable set to **`null`**
  - That way the code can test whether or not a node is the last node
  - Note: This is tested using **`==`**, not the **`equals`** method

Traversing a Linked List

- If a linked list already contains nodes, it can be traversed as follows:
  - Set a local variable equal to the value stored by the head node (its reference)
  - This will provides the location of the first node
  - After accessing the first node, the accessor method for the link instance variable will provide the location of the next node
  - Repeat this until the location of the next node is equal to `null`

Traversing a Linked List

Display 15.4 Traversing a Linked List

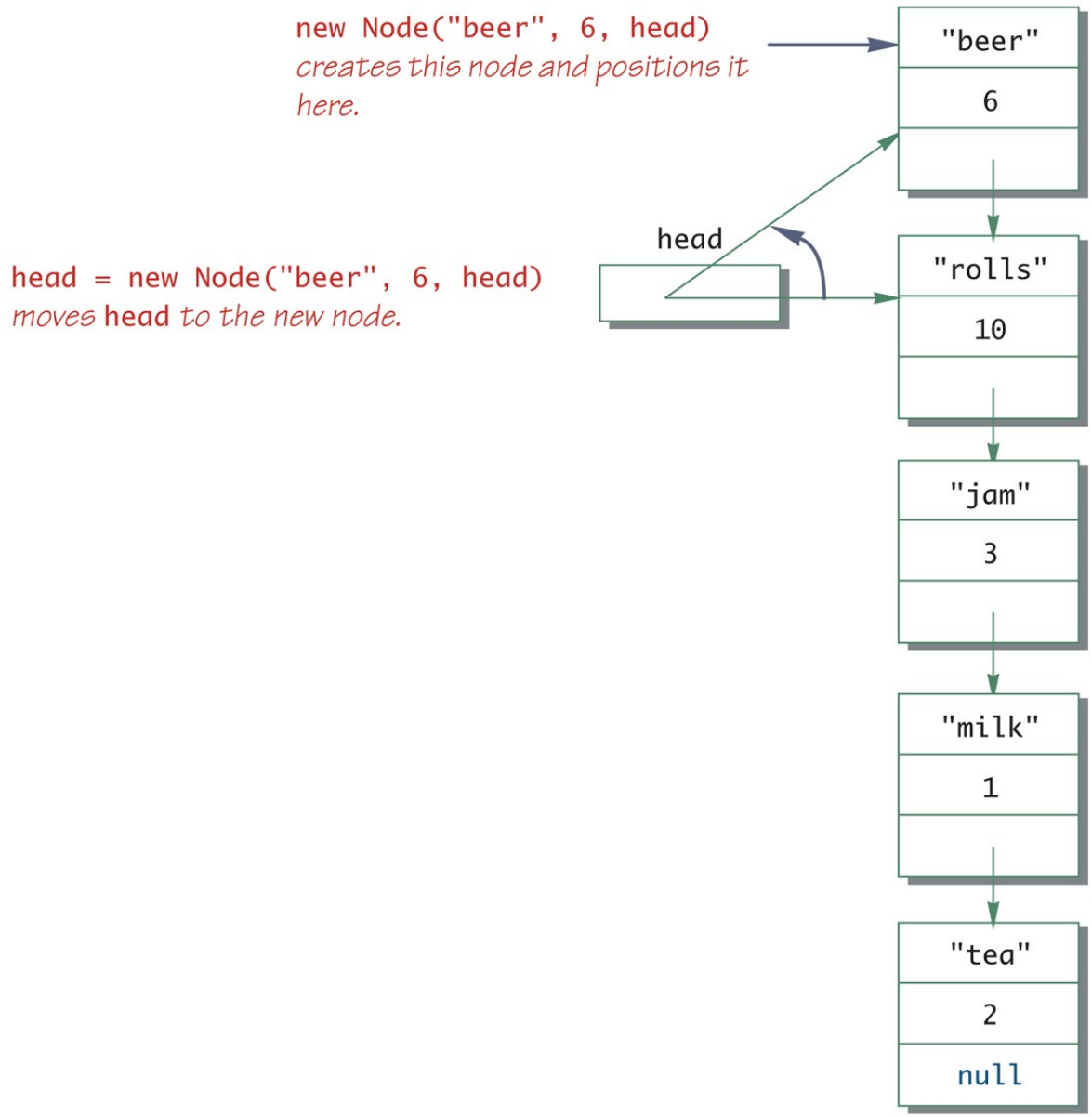


Adding a Node to a Linked List

- The method **`add`** adds a node to the start of the linked list
  - This makes the new node become the first node on the list
- The variable **`head`** gives the location of the current first node of the list
  - Therefore, when the new node is created, its **`link`** field is set equal to **`head`**
  - Then **`head`** is set equal to the new node

A

Display 15.5 Adding a Node at the Start



Deleting the Head Node from a Linked List

- The method **`deleteHeadNode`** removes the first node from the linked list
  - It leaves the **`head`** variable pointing to (i.e., containing a reference to) the old second node in the linked list
- The deleted node will automatically be collected and its memory recycled, along with any other nodes that are no longer accessible
  - In Java, this process is called *automatic garbage collection*

A Linked List Demonstration (Part 1 of 3)

Display 15.6 A Linked List Demonstration

```
1   public class LinkedList1Demo
2   {
3       public static void main(String[] args)
4       {
5           LinkedList1 list = new LinkedList1();
6           list.addToStart("Apples", 1);
7           list.addToStart("Bananas", 2);
8           list.addToStart("Cantaloupe", 3);
9           System.out.println("List has " + list.size()
10                                 + " nodes.");
11          list.outputList();
12          if (list.contains("Cantaloupe"))
13              System.out.println("Cantaloupe is on list.");
```

Cantaloupe *is now in the head node.*

(continued)

A Linked List Demonstration (Part 2 of 3)

Display 15.6 A Linked List Demonstration

```
14            else
15                System.out.println("Cantaloupe is NOT on list.");

16            list.deleteHeadNode();

17            if (list.contains("Cantaloupe"))
18                System.out.println("Cantaloupe is on list.");
19            else
20                System.out.println("Cantaloupe is NOT on list.");

21            while (list.deleteHeadNode())
22                ; //Empty loop body

23            System.out.println("Start of list:");
24            list.outputList();
25            System.out.println("End of list.");
26        }
27    }
```

Removes the head node.

Empties the list. There is no loop body because the method `deleteHeadNode` *both performs an action on the list and returns a Boolean value.*

(continued)

A Linked List Demonstration (Part 3 of 3)

Display 15.6 A Linked List Demonstration

SAMPLE DIALOGUE

```
List has 3 entries.
Cantaloupe 3
Bananas 2
Apples 1
Cantaloupe is on list.
Cantaloupe is NOT on list.
Start of list:
End of list.
```

Node Inner Classes

- Note that the linked list class discussed so far is dependent on an external node class
- A linked list or similar data structure can be made self-contained by making the node class an inner class
- A node inner class so defined should be made private, unless used elsewhere
  - This can simplify the definition of the node class by eliminating the need for accessor and mutator methods
  - Since the instance variables are private, they can be accessed directly from methods of the outer class without causing a privacy leak

Pitfall: Privacy Leaks

- The original node and linked list classes examined so far have a dangerous flaw
  - The node class accessor method returns a reference to a node
  - Recall that if a method returns a reference to an instance variable of a mutable class type, then the **`private`** restriction on the instance variables can be easily defeated
  - The easiest way to fix this problem would be to make the node class a private inner class in the linked list class

A Linked List Class with a Node Inner Class (Part 1 of 6)

Display 15.7 A Linked List Class with a Node Inner Class

```
1  public class LinkedList2
2  {
3      private class Node
4      {
5          private String item;
6          private Node link;
7          public Node()
8          {
9               item = null;
10              link = null;
11         }
12         public Node(String newItem, Node linkValue)
13         {
14             item = newItem;
15             link = linkValue;
16         }
17      }//End of Node inner class
```

It makes no difference whether we make the instance variables of Node *public or private.*

An inner class for the node class

(continued)

A Linked List Class with a Node Inner Class (Part 2 of 6)

Display 15.7 A Linked List Class with a Node Inner Class

```
18      private Node head;

19      public LinkedList2()
20      {
21          head = null;
22      }
23      /**
24       Adds a node at the start of the list with the specified data.
25       The added node will be the first node in the list.
26      */
27      public void addToStart(String itemName)
28      {
29          head = new Node(itemName, head);
30      }

31      /**
32       Removes the head node and returns true if the list contained at least
33       one node. Returns false if the list was empty.
34      */
```

We have simplified this class and the previous linked list class to keep them relatively short. Among other things, these classes should have a copy constructor, an `equals` *method, and a* `clone` *method. Our next linked list example includes these items.*

(continued)

A Linked List Class with a Node Inner Class (Part 3 of 6)

Display 15.7 A Linked List Class with a Node Inner Class

```
35      public boolean deleteHeadNode()
36      {
37          if (head != null)
38          {
39              head = head.link;
40              return true;
41          }
42          else
43              return false;
44      }

45      /**
46       Returns the number of nodes in the list.
47      */
48      public int size()
49      {
50          int count = 0;
51          Node position = head;
```

(continued)

A Linked List Class with a Node Inner Class (Part 4 of 6)

Display 15.7 A Linked List Class with a Node Inner Class

```
52          while (position != null)
53          {
54              count++;
55              position = position.link;
56          }
57          return count;
58      }

59      public boolean contains(String item)
60      {
61          return (find(item) != null);
62      }
```

Note that the outer class has direct access to the inner class's instance variables, such as `link`.

(continued)

A Linked List Class with a Node Inner Class (Part 5 of 6)

Display 15.7 A Linked List Class with a Node Inner Class

```
63      /**
64       Finds the first node containing the target item, and returns a
65       reference to that node. If target is not in the list, null is returned.
66      */
67      private Node find(String target)
68      {
69          Node position = head;
70          String itemAtPosition;
71          while (position != null)
72          {
73              itemAtPosition = position.item;
74              if (itemAtPosition.equals(target))
75                  return position;
76              position = position.link;
77          }
78          return null; //target was not found
79      }
```

(continued)

A Linked List Class with a Node Inner Class (Part 6 of 6)

Display 15.7 A Linked List Class with a Node Inner Class

```
80      public void outputList()
81      {
82          Node position = head;
83          while (position != null)
84          {
85              System.out.println(position.item );
86              position = position.link;
87          }
88      }

89      public boolean isEmpty()
90      {
91          return (head == null);
92      }

93      public void clear()
94      {
95          head = null;
96      }
97  }
```

A Generic Linked List

- A linked list can be created whose **`Node`** class has a *type parameter* ***`T`*** for the type of data stored in the node
  - Therefore, it can hold objects of any class type, including types that contain multiple instance variable
  - The type of the actual object is plugged in for the type parameter ***`T`***
- For the most part, this class can have the same methods, coded in basically the same way, as the previous linked list example
  - The only difference is that a type parameter is used instead of an actual type for the data in the node
- Other useful methods can be added as well

A Generic Linked List Class (Part 1 of 9)

Display 15.8 A Generic Linked List Class

```
1  public class LinkedList3<T>
2  {
3      private class Node<T>
4      {
5          private T data;
6          private Node<T> link;

7          public Node()
8          {
9               data = null;
10              link = null;
11         }

12         public Node(T newData, Node<T> linkValue)
13         {
14             data = newData;
15             link = linkValue;
16         }
17     }//End of Node<T> inner class
```

This linked list holds objects of type `T`. *The type* `T` *should have well-defined* `equals` *and* `toString` *methods.*

(continued)

A Generic Linked List Class (Part 2 of 9)

Display 15.8 A Generic Linked List Class

```
18      private Node<T> head;

19      public LinkedList3()
20      {
21          head = null;
22      }

23      /**
24       Adds a node at the start of the list with the specified data.
25       The added node will be the first node in the list.
26      */
27      public void addToStart(T itemData)
28      {
29          head = new Node<T>(itemData, head);
30      }
```

(continued)

A Generic Linked List Class (Part 3 of 9)

Display 15.8 A Generic Linked List Class

```
31      /**
32       Removes the head node and returns true if the list contained at least
33       one node. Returns false if the list was empty.
34      */
35      public boolean deleteHeadNode()
36      {
37          if (head != null)
38          {
39              head = head.link;
40              return true;
41          }
42          else
43              return false;
44      }
```

(continued)

A Generic Linked List Class (Part 4 of 9)

Display 15.8 A Generic Linked List Class

```
45      /**
46       Returns the number of nodes in the list.
47      */
48      public int size()
49      {
50          int count = 0;
51          Node<T> position = head;
52          while (position != null)
53          {
54              count++;
55              position = position.link;
56          }
57          return count;
58      }

59      public boolean contains(T item)
60      {
61          return (find(item) != null);
62      }
```

(continued)

A Generic Linked List Class (Part 5 of 9)

Display 15.8 A Generic Linked List Class

```
63      /**
64       Finds the first node containing the target item, and returns a
65       reference to that node. If target is not in the list, null is returned.
66      */
67      private Node<T> find(T target)
68      {
69          Node<T> position = head;
70          T itemAtPosition;
71          while (position != null)
72          {
73              itemAtPosition = position.data;
74              if (itemAtPosition.equals(target))
75                  return position;
76              position = position.link;
77          }
78          return null; //target was not found
79      }
```

Type T *must have a well-defined* equals *method for this to work.*

(continued)

A Generic Linked List Class (Part 6 of 9)

Display 15.8 A Generic Linked List Class

```
80      /**
81       Finds the first node containing the target and returns a reference
82        to the data in that node. If target is not in the list, null is returned.
83      */
84      public T findData(T target)
85      {
86          return find(target).data;
87      }

88      public void outputList()
89      {
90          Node<T> position = head;
91          while (position != null)
92          {
93              System.out.println(position.data);
94              position = position.link;
95          }
96      }
```

Type T *must have a well-defined* toString *method for this to work.*



(continued)

A Generic Linked List Class (Part 7 of 9)

Display 15.8 A Generic Linked List Class

```
 97      public boolean isEmpty()
 98      {
 99          return (head == null);
100      }

101      public void clear()
102      {
103          head = null;
104      }
```

(continued)

A Generic Linked List Class (Part 8 of 9)

Display 15.8 A Generic Linked List Class

```
105     /*
106      For two lists to be equal they must contain the same data items in
107      the same order. The equals method of T is used to compare data items.
108     */
109     public boolean equals(Object otherObject)
110      {
111          if (otherObject == null)
112              return false;
113          else if (getClass() != otherObject.getClass())
114              return false;
115          else
116          {
117              LinkedList3<T> otherList = (LinkedList3<T>)otherObject;
```

(continued)

A Generic Linked List Class (Part 9 of 9)

Display 15.8 A Generic Linked List Class

```
118             if (size() != otherList.size())
119                 return false;
120             Node<T> position = head;
121             Node<T> otherPosition = otherList.head;
122             while (position != null)
123             {
124                 if (!(position.data.equals(otherPosition.data)))
125                     return false;
126                 position = position.link;
127                 otherPosition = otherPosition.link;
128             }
129             return true; //no mismatch was not found
130         }
131     }
132 }
```

A Sample Class for the Data in a Generic Linked List (Part 1 of 2)

Display 15.9 A Sample Class for the Data in a Generic Linked List

```
1   public class Entry
2   {
3       private String item;
4       private int count;

5       public Entry(String itemData, int countData)
6       {
7           item = itemData;
8           count = countData;
9       }

10      public String toString()
11      {
12          return (item + " " + count);
13      }
```

(continued)

A Sample Class for the Data in a Generic Linked List (Part 2 of 2)

Display 15.9 A Sample Class for the Data in a Generic Linked List

```
14      public boolean equals(Object otherObject)
15      {
16          if (otherObject == null)
17              return false;
18          else if (getClass() != otherObject.getClass())
19              return false;
20          else
21          {
22              Entry otherEntry = (Entry)otherObject;
23              return (item.equals(otherEntry.item)
24                      && (count == otherEntry.count));
25          }
26      }

        <There should be other constructors and methods, including accessor and
                mutator methods, but we do not use them in this demonstration.>
27  }
```

A Generic Linked List Demonstration (Part 1 of 2)

Display 15.10 A Generic Linked List Demonstration

```
1  public class GenericLinkedListDemo
2  {
3      public static void main(String[] args)
4      {
5          LinkedList3<Entry> list = new LinkedList3<Entry>();
6
7          Entry entry1 = new Entry("Apples", 1);
8          list.addToStart(entry1);
9          Entry entry2 = new Entry("Bananas", 2);
10         list.addToStart(entry2);
11         Entry entry3 = new Entry("Cantaloupe", 3);
12         list.addToStart(entry3);
13         System.out.println("List has " + list.size()
14                            + " nodes.");
```

(continued)

A Generic Linked List Demonstration (Part 2 of 2)

Display 15.10 A Generic Linked List Demonstration

```
15            list.outputList();
16            System.out.println("End of list.");
17        }
18    }
```

SAMPLE DIALOGUE

```
List has 3 nodes.
Cantaloupe 3
Bananas 2
Apples 1
End of list.
```

Pitfall: Using `Node` instead of `Node<T>`

- **Note:** This pitfall is explained by example – any names can be substituted for the node `Node` and its parameter `<T>`
- When defining the `LinkedList3<T>` class, the type for a node is `Node<T>`, not `Node`
  - If the `<T>` is omitted, this is an error for which the compiler may or may not issue an error message (depending on the details of the code), and even if it does, the error message may be quite strange
  - Look for a missing `<T>` when a program that uses nodes with type parameters gets a strange error message or doesn't run correctly

A Generic Linked List: the `equals` Method

- Like other classes, a linked list class should normally have an `equals` method
- The `equals` method can be defined in a number of reasonable ways
  - Different definitions may be appropriate for different situations
- Two such possibilities are the following:
  1. They contain the same data entries (possibly in different orders)
  2. They contain the same data entries in the same order
- Of course, the type plugged in for `T` must also have redefined the `equals` method

An `equals` Method for the Linked List in Display 15.7 (Part 1 of 2)

Display 15.11 An equals Method for the Linked List in Display 15.7

```
1       /*
2        For two lists to be equal they must contain the same data items in
3        the same order.
4       */
5       public boolean equals(Object otherObject)
6        {
7           if (otherObject == null)
8               return false;
9           else if (getClass() != otherObject.getClass())
10              return false;
11          else
12          {
13              LinkedList2 otherList = (LinkedList2)otherObject;
```

(continued)

An `equals` Method for the Linked List in Display 15.7 (Part 2 of 2)

Display 15.11 An equals Method for the Linked List in Display 15.7

```
14              if (size() != otherList.size())
15                  return false;
16              Node position = head;
17              Node otherPosition = otherList.head;
18              while (position != null)
19              {
20                  if ( (!(position.item.equals(otherPosition.item))))
21                     return false;
22                  position = position.link;
23                  otherPosition = otherPosition.link;
24              }
25              return true; //A mismatch was not found
26          }
27      }
```

Simple Copy Constructors and `clone` Methods

- There is a simple way to define copy constructors and the `clone` method for data structures such as linked lists
  - Unfortunately, this approach produces only shallow copies
- The private helping method `copyOf` is used by both the copy constructor and the `clone` method
- The copy constructor uses `copyOf` to create a copy of the list of nodes
- The `clone` method first invokes its superclass `clone` method, and then uses `copyOf` to create a clone of the list of nodes

A Generic Linked List: the private method `copyOf`

- The private helping method `copyOf` takes an argument that is a reference to a head node of a linked list, and returns a reference to the head node of a copy of that list
  - It goes down the argument list one node at a time and makes a copy of each node
  - The new nodes are added to the end of the linked list being built
- However, although this produces a new linked list with all new nodes, the new list is not truly independent because the data object is not cloned

A Copy Constructor and `clone` Method for a Generic Linked List (Part 1 of 6)

Display 15.12 A Copy Constructor and `clone` Method for a Generic Linked List

```
1  public class LinkedList3<T> implements Cloneable
2  {
3      private class Node<T>
4      {
5          private T data;
6          private Node<T> link;

7          public Node()
8          {
9              data = null;
10             link = null;
11         }
```

This copy constructor and this `clone` method do not make deep copies. We discuss one way to make a deep copy in the Programming Tip subsection "Use a Type Parameter Bound for a Better `clone`."

(continued)

A Copy Constructor and `clone` Method for a Generic Linked List (Part 2 of 6)

Display 15.12 A Copy Constructor and `clone` Method for a Generic Linked List

```
12          public Node(T newData, Node<T> linkValue)
13          {
14              data = newData;
15              link = linkValue;
16          }
17      }//End of Node<T> inner class

18      private Node<T> head;
```

<All the methods from Display 15.8 are in the class definition,
but they are not repeated in this display.>

(continued)

A Copy Constructor and `clone` Method for a Generic Linked List (Part 3 of 6)

Display 15.12 A Copy Constructor and `clone` Method for a Generic Linked List

```
19      /**
20       Produces a new linked list, but it is not a true deep copy.
21       Throws a NullPointerException if other is null.
22     */
23      public LinkedList3(LinkedList3<T> otherList)
24      {
25          if (otherList == null)
26              throw new NullPointerException( );
27          if (otherList.head == null)
28              head = null;
29          else
30              head = copyOf(otherList.head);
31      }
32
33
```

(continued)

A Copy Constructor and `clone` Method for a Generic Linked List (Part 4 of 6)

Display 15.12 A Copy Constructor and `clone` Method for a Generic Linked List

```
34      public LinkedList3<T> clone( )
35      {
36          try
37          {
38              LinkedList3<T> copy =
39                                  (LinkedList3<T>)super.clone( );
40              if (head == null)
41                  copy.head = null;
42              else
43                  copy.head = copyOf(head);
44              return copy;
45          }
46          catch(CloneNotSupportedException e)
47          {//This should not happen.
48              return null; //To keep the compiler happy.
49          }
50      }
```

(continued)

A Copy Constructor and `clone` Method for a Generic Linked List (Part 5 of 6)

Display 15.12 A Copy Constructor and `clone` Method for a Generic Linked List

```
51      /*
52        Precondition: otherHead != null
53        Returns a reference to the head of a copy of the list
54        headed by otherHead. Does not return a true deep copy.
55      */
56      private Node<T> copyOf(Node<T> otherHead)
57      {
58          Node<T> position = otherHead;//moves down other's list.
59          Node<T> newHead; //will point to head of the copy list.
60          Node<T> end = null; //positioned at end of new growing list.
```

(continued)

A Copy Constructor and `clone` Method for a Generic Linked List (Part 6 of 6)

Display 15.12 A Copy Constructor and `clone` Method for a Generic Linked List

```
61          //Create first node:
62          newHead =
63                new Node<T>(position.data, null);
64          end = newHead;
65          position = position.link;

66          while (position != null)
67          {//copy node at position to end of new list.
68              end.link =
69                  new Node<T>(position.data, null);
70              end = end.link;
71              position = position.link;
72          }

73          return newHead;
74      }
75  }
```

Invoking `clone` *with* `position.data` *would be illegal.*

Invoking `clone` *with* `position.data` *would be illegal.*

Pitfall: The `clone` Method Is Protected in Object

- It would have been preferable to clone the data belonging to the list being copied in the `copyOf` method as follows:

```
nodeReference = new
    Node((T)(position.data).clone(), null);
```

- However, this is not allowed, and this code will not compile
  - The error message generated will state that `clone` is protected in `Object`
  - Although the type used is `T`, not `Object`, any class can be plugged in for `T`
  - When the class is compiled, all that Java knows is that `T` is a descendent class of Object

Exceptions

- A generic data structure is likely to have methods that throw exceptions
- Situations such as a **`null`** argument to the copy constructor may be handled differently in different situations
  - If this happens, it is best to throw a **`NullPointerException`**, and let the programmer who is using the linked list handle the exception, rather than take some arbitrary action
  - A **`NullPointerException`** is an *unchecked* exception: it need not be caught or declared in a throws clause

Tip: Use a Type Parameter Bound for a Better `clone`

- One solution to this problem is to place a bound on the type parameter `T` so that it must satisfy a suitable interface
  - Although there is no standard interface that does this, it is easy to define one
- For example, a `PubliclyCloneable` interface could be defined

Tip: Use a Type Parameter Bound for a Better `clone`

- Any class that implements the **`PubliclyCloneable`** interface would have these three properties:
  1. It would implement the **`Cloneable`** interface because **`PubliclyCloneable`** extends **`Cloneable`**
  2. It would have to implement a public **`clone`** method
  3. Its **`clone`** method would have to make a deep copy

The `PubliclyCloneable` Interface

Display 15.13 The `PubliclyCloneable` Interface

```
1  /*
2   The programmer who defines a class implementing this interface
3   has the responsibility to define clone so it makes a deep copy
4   (in the officially sectioned way.)
5  */

6  public interface PubliclyCloneable extends Cloneable
7  {
8      public Object clone();
9  }
```

Any class that implements `PubliclyCloneable` *automatically implements* `Cloneable`.

Any class that implements `PubliclyCloneable` *must have a public* `clone` *method.*

A Generic Linked List with a Deep Copy `clone` Method (Part 1 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

```
1  public class LinkedList<T extends PubliclyCloneable>
2                                  implements PubliclyCloneable
3  {
4      private class Node<T>
5      {
6          private T data;
7          private Node<T> link;

8          public Node()
9          {
10             data = null;
11             link = null;
12         }
```

(continued)

A Generic Linked List with a Deep Copy `clone` Method (Part 2 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

```
13          public Node(T newData, Node<T> linkValue)
14          {
15              data = newData;
16              link = linkValue;
17          }
18       }//End of Node<T> inner class

19      private Node<T> head;

20      public LinkedList()
21      {
22          head = null;
23      }
```

(continued)

A Generic Linked List with a Deep Copy `clone` Method (Part 3 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

```
24        /**
25        Produces a new linked list, but it is not a true deep copy.
26        Throws a NullPointerException if other is null.
27      */
28      public LinkedList(LinkedList<T> otherList)
29      {
30          if (otherList == null)
31              throw new NullPointerException( );
32          if (otherList.head == null)
33              head = null;
34          else
35              head = copyOf(otherList.head);
36      }
37
```

(continued)

A Generic Linked List with a Deep Copy `clone` Method (Part 4 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

```
38      public LinkedList<T> clone( )
39      {
40          try
41          {
42              LinkedList<T> copy =
43                                  (LinkedList<T>)super.clone( );
44              if (head == null)
45                  copy.head = null;
46              else
47                  copy.head = copyOf(head);
48              return copy;
49          }
50          catch(CloneNotSupportedException e)
51          {//This should not happen.
52              return null; //To keep the compiler happy.
53          }
54      }
```

(continued)

A Generic Linked List with a Deep Copy `clone` Method (Part 5 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

This definition of `copyOf` *gives a deep copy of the linked list.*

```
55        /*
56          Precondition: otherHead != null
57          Returns a reference to the head of a copy of the list
58          headed by otherHead. Returns a true deep copy.
59        */
60        private Node<T> copyOf(Node<T> otherHead)
61        {
62            Node<T> position = otherHead;//moves down other's list.
63            Node<T> newHead; //will point to head of the copy list.
64            Node<T> end = null; //positioned at end of new growing list.

65            //Create first node:
66            newHead =
67                  new Node<T>((T)(position.data).clone(), null);
68                  end = newHead;
69            position = position.link;
```

(continued)

A Generic Linked List with a Deep Copy `clone` Method (Part 6 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

```
70          while (position != null)
71          {//copy node at position to end of new list.
72              end.link =
73                  new Node<T>((T)(position.data).clone(), null);
74              end = end.link;
75              position = position.link;
76          }

77          return newHead;
78      }
79
```

(continued)

A Generic Linked List with a Deep Copy `clone` Method (Part 7 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

```
80      public boolean equals(Object otherObject)
81      {
82          if (otherObject == null)
83              return false;
84          else if (getClass() != otherObject.getClass())
85              return false;
86          else
87          {
88              LinkedList<T> otherList = (LinkedList<T>)otherObject;
```

<The rest of the definition is the same as in Display 15.8. The only difference between this definition of `equals` and the one in Display 15.8 is that we have replaced the class name `LinkedList3<T>` with `LinkedList<T>`.>

```
89      }
```

(continued)

A Generic Linked List with a Deep Copy `clone` Method (Part 8 of 8)

Display 15.14 A Generic Linked List with a Deep Copy `clone` Method

<All the other methods from Display 15.8 are in the class definition,
but are not repeated in this display. >

```
90        public String toString()
91        {
92            Node<T> position = head;
93            String theString = "";
94            while (position != null)
95            {
96                theString = theString + position.data + "\n";
97                position = position.link;
98            }
99            return theString;
100       }
101   }
```

We added a `toString` method so `LinkedList<T>` would have all the properties we want `T` to have.

A Linked List with a Deep Copy `clone` Method

- Some of the details of the clone method in the previous linked list class may be puzzling, since the following code would also return a deep copy:

```
public LinkedList<T> clone()
{
  return new LinkedList<T>(this);
}
```

- However, because the class implements **`PubliclyCloneable`** which, in turn, extends **`Cloneable`**, it must implement the **`Cloneable`** interface as specified in the Java documentation

A PubliclyCloneable Class (Part 1 of 4)

Display 15.15 A PubliclyCloneable Class

```
1   public class StockItem implements PubliclyCloneable
2   {
3       private String name;
4       private int number;

5       public StockItem()
6       {
7           name = null;
8           number = 0;
9       }

10      public StockItem(String nameData, int numberData)
11      {
12          name = nameData;
13          number = numberData;
14      }
```

(continued)

A PubliclyCloneable Class (Part 2 of 4)

Display 15.15 A PubliclyCloneable Class

```
15      public void setNumber(int newNumber)
16      {
17          number = newNumber;
18      }

19      public void setName(String newName)
20      {
21          name = newName;
22      }

23      public String toString()
24      {
25          return (name + " " + number);
26      }
```

(continued)

A PubliclyCloneable Class (Part 3 of 4)

Display 15.15 A PubliclyCloneable Class

```
27      public Object clone()
28      {
29         try
30         {
31            return super.clone();
32         }
33         catch(CloneNotSupportedException e)
34         {//This should not happen.
35            return null; //To keep compiler happy.
36         }
37      }
38
```

(continued)

A PubliclyCloneable Class (Part 4 of 4)

Display 15.15 A PubliclyCloneable Class

```
39      public boolean equals(Object otherObject)
40      {
41          if (otherObject == null)
42              return false;
43          else if (getClass() != otherObject.getClass())
44              return false;
45          else
46          {
47              StockItem otherItem = (StockItem) otherObject;
48              return (name.equalsIgnoreCase(otherItem.name)
49                      && number == otherItem.number);
50          }
51      }
52  }
```

Demonstration of Deep Copy clone (Part 1 of 3)

Display 15.16 Demonstration of Deep Copy clone

```
1  public class DeepDemo
2  {
3      public static void main(String[] args)
4      {
5          LinkedList<StockItem> originalList =
6                                      new LinkedList<StockItem>();
7          originalList.addToStart(new StockItem("red dress", 1));
8          originalList.addToStart(new StockItem("black shoe", 2));

9          LinkedList<StockItem> copyList = originalList.clone();
10         if (originalList.equals(copyList))
11             System.out.println("OK, Lists are equal.");
```

(continued)

Demonstration of Deep Copy clone (Part 2 of 3)

Display 15.16 Demonstration of Deep Copy clone

```
12          System.out.println("Now we change copyList.");
13          StockItem dataEntry =
14                    copyList.findData(new StockItem("red dress", 1));
15          dataEntry.setName("orange pants");

16          System.out.println("originalList:");
17          originalList.outputList();

18          System.out.println("copyList:");
19          copyList.outputList();
20
21          System.out.println("Only one list is changed.");
22      }
23  }
```

(continued)

Demonstration of Deep Copy clone (Part 3 of 3)

Display 15.16 Demonstration of Deep Copy clone

SAMPLE DIALOGUE

```
OK, Lists are equal.
Now we change copyList.
originalList:
black shoe 2
red dress 1
copyList:
black shoe 2
orange pants 1
Only one list is changed.
```

Tip: Cloning is an "All or Nothing" Affair

- If a **`clone`** method is defined for a class, then it should follow the official Java guidelines
  - In particular, it should implement the **`Cloneable`** interface

Iterators

- A collection of objects, such as the nodes of a linked list, must often be traversed in order to perform some action on each object
  - An *iterator* is any object that enables a list to be traversed in this way
- A linked list class may be created that has an iterator inner class
  - If iterator variables are to be used outside the linked list class, then the iterator class would be made public
  - The linked list class would have an **`iterator`** method that returns an iterator for its calling object
  - Given a linked list named **`list`**, this can be done as follows:

```
LinkedList2.List2Iterator i = list.iterator();
```

Iterators

- The basic methods used by an iterator are as follows:
  - **`restart`**: Resets the iterator to the beginning of the list
  - **`hasNext`**: Determines if there is another data item on the list
  - **`next`**: Produces the next data item on the list

A Linked List with an Iterator (Part 1 of 6)

Display 15.17 A Linked List with an Iterator

```
1   import java.util.NoSuchElementException;

2   public class LinkedList2
3   {
4       private class Node
5       {
6           private String item;
7           private Node link;

            <The rest of the definition of the Node inner class is given in Display 15.7.>

8       }//End of Node inner class
```

This is the same as the class in Displays 15.7 and 15.11 except that the `List2Iterator` *inner class and the* `iterator()` *method have been added.*

(continued)

A Linked List with an Iterator (Part 2 of 6)

Display 15.17 A Linked List with an Iterator

```
 9       /**
10        If the list is altered any iterators should invoke restart or
11        the iterator's behavior may not be as desired.
12       */
13       public class List2Iterator
14       {
15           private Node position;
16           private Node previous;//previous value of position

17           public List2Iterator()
18           {
19               position = head; //Instance variable head of outer class.
20               previous = null;
21           }

22           public void restart()
23           {
24               position = head; //Instance variable head of outer class.
25               previous = null;
26           }
```

An inner class for iterators for `LinkedList2`.

(continued)

A Linked List with an Iterator (Part 3 of 6)

Display 15.17 A Linked List with an Iterator

```
27          public String next()
28          {
29              if (!hasNext())
30                  throw new NoSuchElementException();

31              String toReturn = position.item;
32              previous = position;
33              position = position.link;
34              return toReturn;
35          }

36          public boolean hasNext()
37          {
38              return (position != null);
39          }
```

(continued)

A Linked List with an Iterator (Part 4 of 6)

Display 15.17 A Linked List with an Iterator

```
40          /**
41           Returns the next value to be returned by next().
42           Throws an IllegalStateExpression if hasNext() is false.
43          */
44          public String peek()
45          {
46               if (!hasNext())
47                  throw new IllegalStateException();
48               return position.item;
49          }
```

(continued)

A Linked List with an Iterator (Part 5 of 6)

Display 15.17 A Linked List with an Iterator

```
50          /**
51           Adds a node before the node at location position.
52           previous is placed at the new node. If hasNext() is
53           false, then the node is added to the end of the list.
54           If the list is empty, inserts node as the only node.
55          */
56          public void addHere(String newData)
              <The rest of the method addHere is Self-Test Exercise 11.>

57          /**
58           Changes the String in the node at location position.
59           Throws an IllegalStateException if position is not at a node,
60          */
61          public void changeHere(String newData)
              <The rest of the method addHere is Self-Test Exercise 13.>
```

(continued)

A Linked List with an Iterator (Part 6 of 6)

Display 15.17 A Linked List with an Iterator

```
62          /**
63           Deletes the node at location position and
64           moves position to the "next" node.
65           Throws an IllegalStateException if the list is empty.
66          */
67          public void delete()
              <The rest of the method delete is Self-Test Exercise 12.>
68        }//End of List2Iterator inner class

69      private Node head;

70      public List2Iterator iterator()
71      {
72          return new List2Iterator();
73      }
    <The other methods and constructors are identical to those in Displays 15.7 and 15.11.>
74  }
```

If `list` *is an object of the class* `LinkedList2`*, then* `list.iterator()` *returns an iterator for* `list`*.*

Using an Iterator (Part 1 of 6)

Display 15.18 Using an Iterator

```
1  public class IteratorDemo
2  {
3      public static void main(String[] args)
4      {
5          LinkedList2 list = new LinkedList2();
6          LinkedList2.List2Iterator i = list.iterator();

7          list.addToStart("shoes");
8          list.addToStart("orange juice");
9          list.addToStart("coat");
```

(continued)

Using an Iterator (Part 2 of 6)

Display 15.18 Using an Iterator

```
10          System.out.println("List contains:");
11          i.restart();
12          while(i.hasNext())
13              System.out.println(i.next());
14          System.out.println();

15          i.restart();
16          i.next();
17          System.out.println("Will delete the node for " + i.peek());
18          i.delete();
```

(continued)

Using an Iterator (Part 3 of 6)

Display 15.18 Using an Iterator

```
19          System.out.println("List now contains:");
20          i.restart();
21          while(i.hasNext())
22              System.out.println(i.next());
23          System.out.println();

24          i.restart();
25          i.next();
26          System.out.println("Will add one node before " + i.peek());
27          i.addHere("socks");
28          System.out.println("List now contains:");
29          i.restart();
30          while(i.hasNext())
31              System.out.println(i.next());
```

(continued)

Using an Iterator (Part 4 of 6)

Display 15.18 Using an Iterator

```
32          System.out.println();
33          System.out.println("Changing all items to credit card.");
34          i.restart();
35          while(i.hasNext())
36          {
37              i.changeHere("credit card");
38              i.next();
39          }
40          System.out.println();

41          System.out.println("List now contains:");
42          i.restart();
43          while(i.hasNext())
44              System.out.println(i.next());
45          System.out.println();
46      }
47  }
```

(continued)

Using an Iterator (Part 5 of 6)

Display 15.18 Using an Iterator

SAMPLE DIALOGUE

```
List contains:
coat
orange juice
shoes

Will delete the node for orange juice
List now contains:
coat
shoes
```

(continued)

Using an Iterator (Part 6 of 6)

Display 15.18 Using an Iterator

```
Will add one node before shoes
List now contains:
coat
socks
shoes

Changing all items to credit card.

List now contains:
credit card
credit card
credit card
```

The Java Iterator Interface

- Java has an interface named `Iterator` that specifies how Java would like an iterator to behave
  - Although the iterators examined so far do not satisfy this interface, they could be easily redefined to do so

Adding and Deleting Nodes

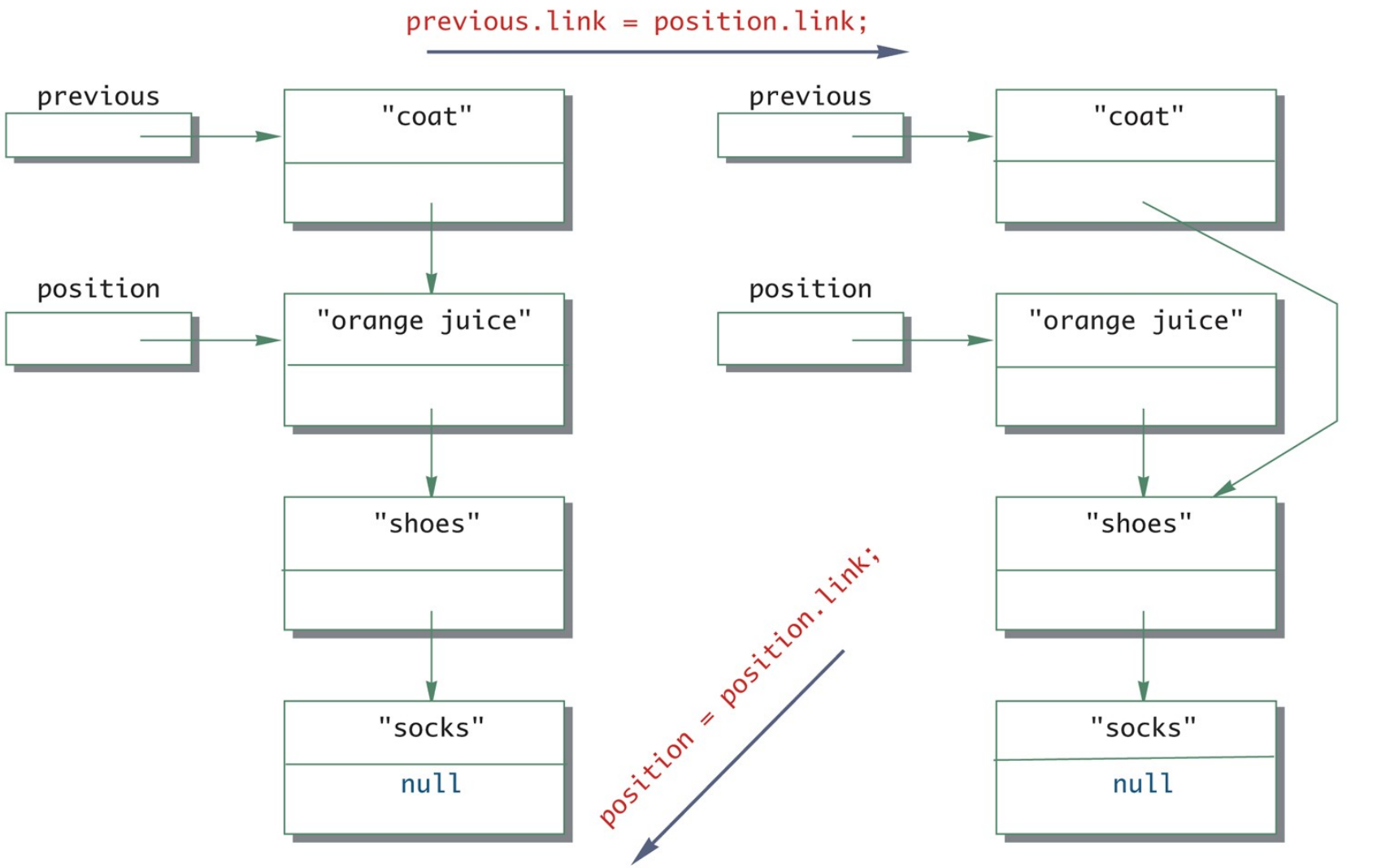
- An iterator is normally used to add or delete a node in a linked list
- Given iterator variables **`position`** and **`previous`**, the following two lines of code will delete the node at location **`position`**:

```
previous.link = position.link;
position = position.link;
```

  - Note: **`previous`** points to the node before **`position`**

Deleting a Node (Part 1 of 2)

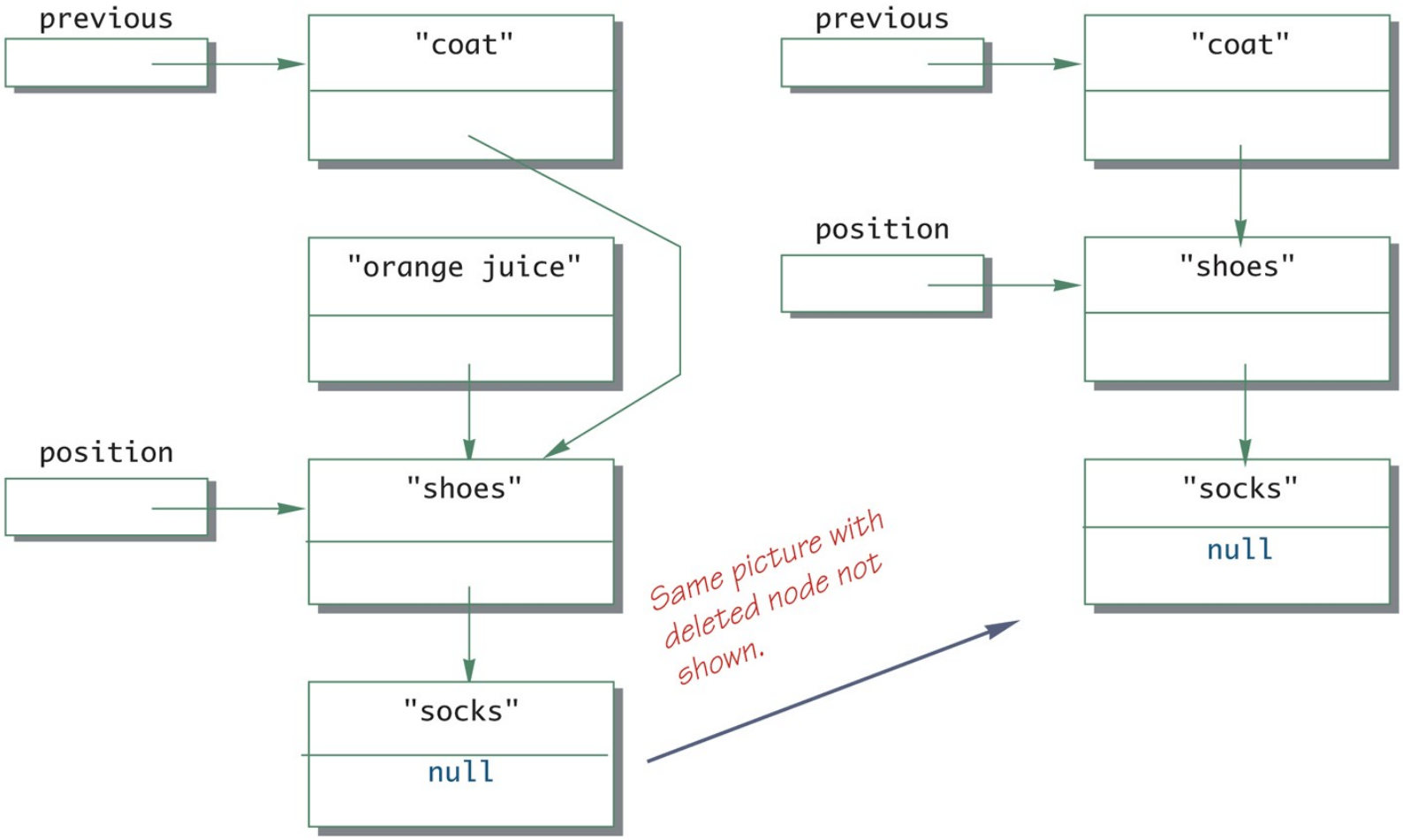
Display 15.19 Deleting a Node



(continued)

Deleting a Node (Part 2 of 2)

Display 15.19 Deleting a Node



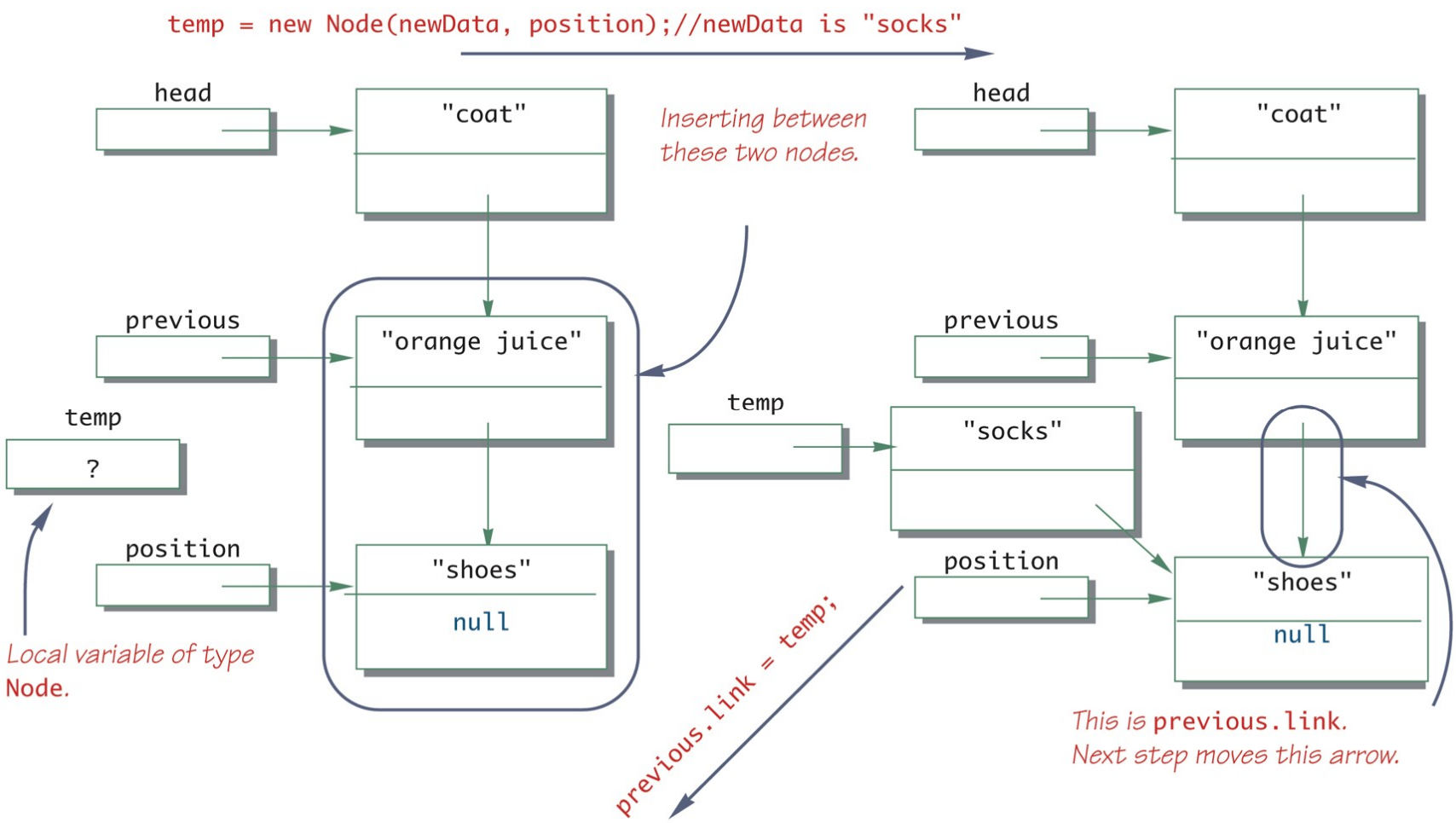
Adding and Deleting Nodes

- Note that Java has *automatic garbage collection*
  - In many other languages the programmer has to keep track of deleted nodes and explicitly return their memory for recycling
  - This procedure is called *explicit memory management*
- The iterator variables **`position`** and **`previous`** can be used to add a node as well
  - **`previous`** will point to the node before the insertion point, and **`position`** will point to the node after the insertion point

```
Node temp = new Node(newData,position);
previous.link = temp;
```

Adding a Node between Two Nodes (Part 1 of 2)

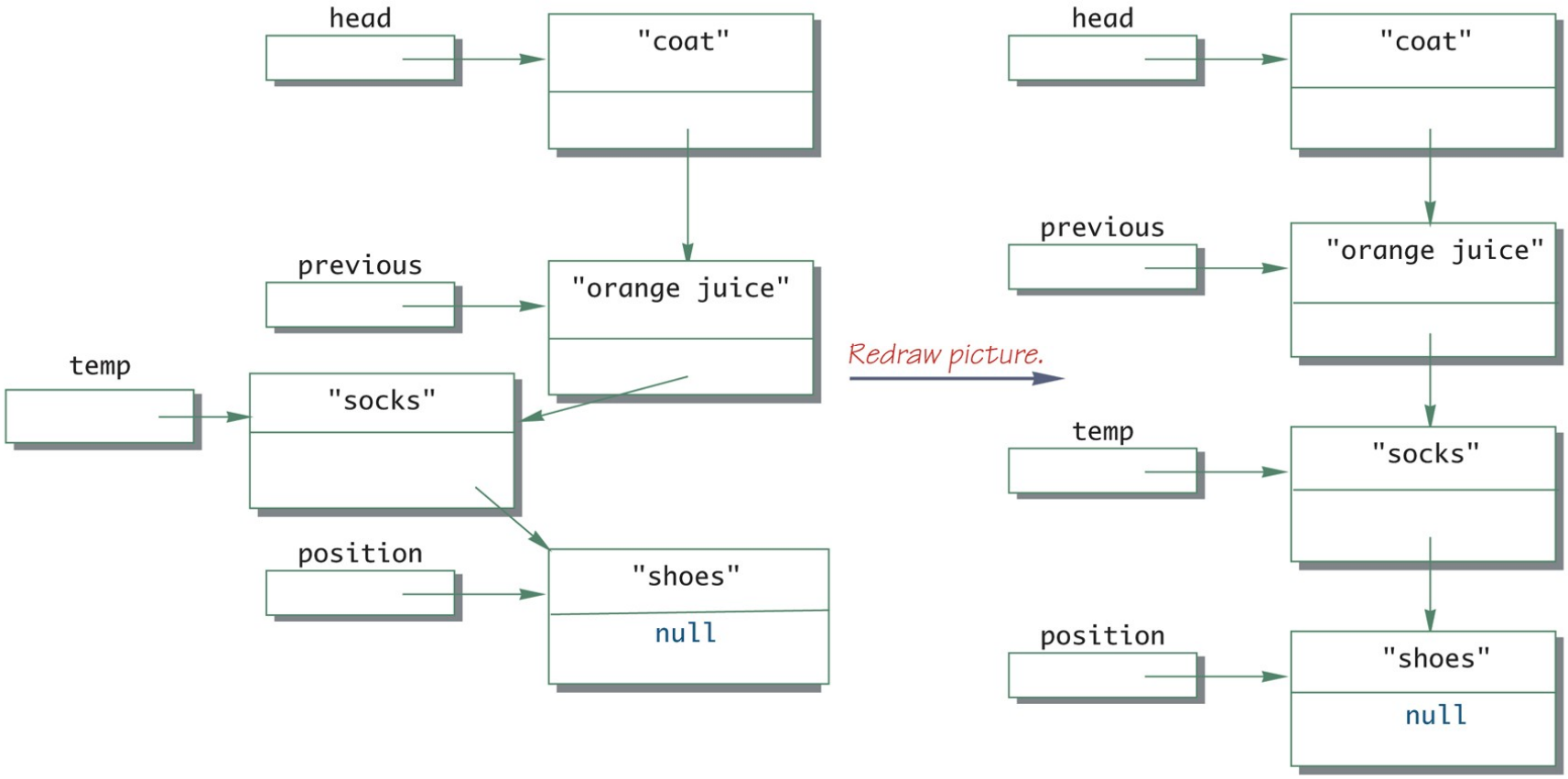
Display 15.20 **Adding a Node between Two Nodes**



(continued)

Adding a Node between Two Nodes (Part 2 of 2)

Display 15.20 Adding a Node between Two Nodes



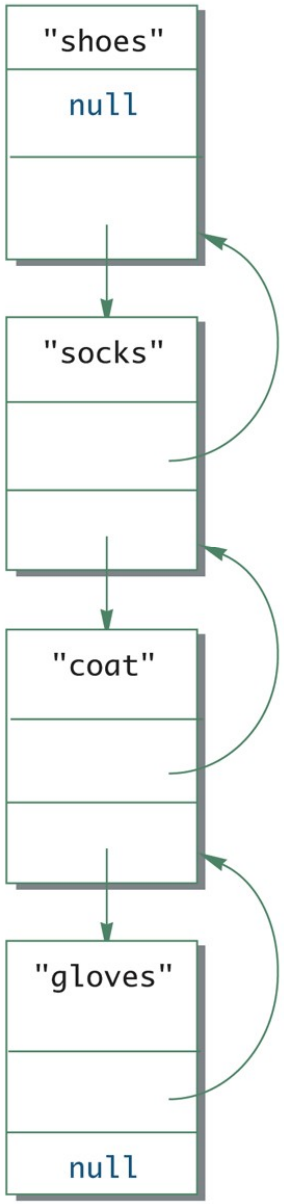
Variations on a Linked List

- An ordinary linked list allows movement in one direction only
- However, a doubly linked list has one link that references the next node, and one that references the previous node
- The node class for a doubly linked list can begin as follows:

```
private class TwoWayNode
{
   private String item;
   private TwoWayNode previous;
   private TwoWayNode next;
   . . .
```

- In addition, the constructors and methods in the doubly linked list class would be modified to accommodate the extra link

Display 15.21 **A Doubly Linked List**



The Stack Data Structure

- A stack data structure is not necessarily a linked data structure, but can be implemented as one
  - A stack is a data structure that removes items in the reverse order of which they were inserted (LIFO: Last In First Out)
  - A linked list that inserts and deletes only at the head of the list is a stack

The Queue Data Structure

- A queue is a data structure that handles data in a first-in/first-out fashion (FIFO) like a line at a bank
  - Customers add themselves to the end of the line and are served from the front of the line
- A queue can be implemented with a linked list
  - However, a queue needs a pointer at both the head and *tail* (the end) of the linked list
  - Nodes are removed from the *front* (head end), and are added to the *back* (tail end)

A Queue Class (Part 1 of 5)

Display 15.22 A Queue Class

```
1  public class Queue
2  {
3      private class Node
4      {
5          private String item;
6          private Node link;

7          public Node()
8          {
9              item = null;
10             link = null;
11         }
```

(continued)

A Queue Class (Part 2 of 5)

Display 15.22 A Queue Class

```
12          public Node(String newItem, Node linkValue)
13          {
14              item = newItem;
15              link = linkValue;
16          }
17       }//End of Node inner class

18       private Node front;
19       private Node back;

20       public Queue()
21       {
22           front = null;
23           back = null;
24       }
```

(continued)

A Queue Class (Part 3 of 5)

Display 15.22 A Queue Class

```
25      /**
26       Adds a String to the back of the queue.
27      */
28      public void addToBack(String itemName)
        <The definition of this method is Self-Test Exercise 14.>

29      public boolean isEmpty()
30      {
31          return (front == null);
32      }

33      public void clear()
34      {
35          front = null;
36          back = null;
37      }
```

(continued)

A Queue Class (Part 4 of 5)

Display 15.22 A Queue Class

```
38      /**
39       Returns the String in the front of the queue.
40       Returns null if queue is empty.
41      */
42      public String whoIsNext()
43      {
44          if (front == null)
45              return null;
46          else
47              return front.item;
48      }
49
```

(continued)

A Queue Class (Part 5 of 5)

Display 15.22 A Queue Class

```
50      /**
51       Removes a String from the front of the queue.
52       Returns false if the list is empty.
53      */
54      public boolean removeFront()
55      {
56          if (front != null)
57          {
58              front = front.link;
59              return true;
60          }
61          else
62              return false;
63      }
64  }
```

Demonstration of the Queue Class (Part 1 of 2)

Display 15.23 Demonstration of the Queue Class

```
1  public class QueueDemo
2  {
3      public static void main(String[] args)
4      {
5          Queue q = new Queue();

6          q.addToBack("Tom");
7          q.addToBack("Dick");
8          q.addToBack("Harriet");
```

Items come out of the queue in the same order that they went into the queue.

(continued)

Demonstration of the Queue Class (Part 2 of 2)

Display 15.23 Demonstration of the Queue Class

```
 9            while(!q.isEmpty())
10            {
11                System.out.println(q.whoIsNext());
12                q.removeFront();
13            }
14            System.out.println("The queue is empty.");
15        }
16    }
```

Items come out of the queue in the same order that they went into the queue.

SAMPLE DIALOGUE

```
Tom
Dick
Harriet
The queue is empty.
```

Trees

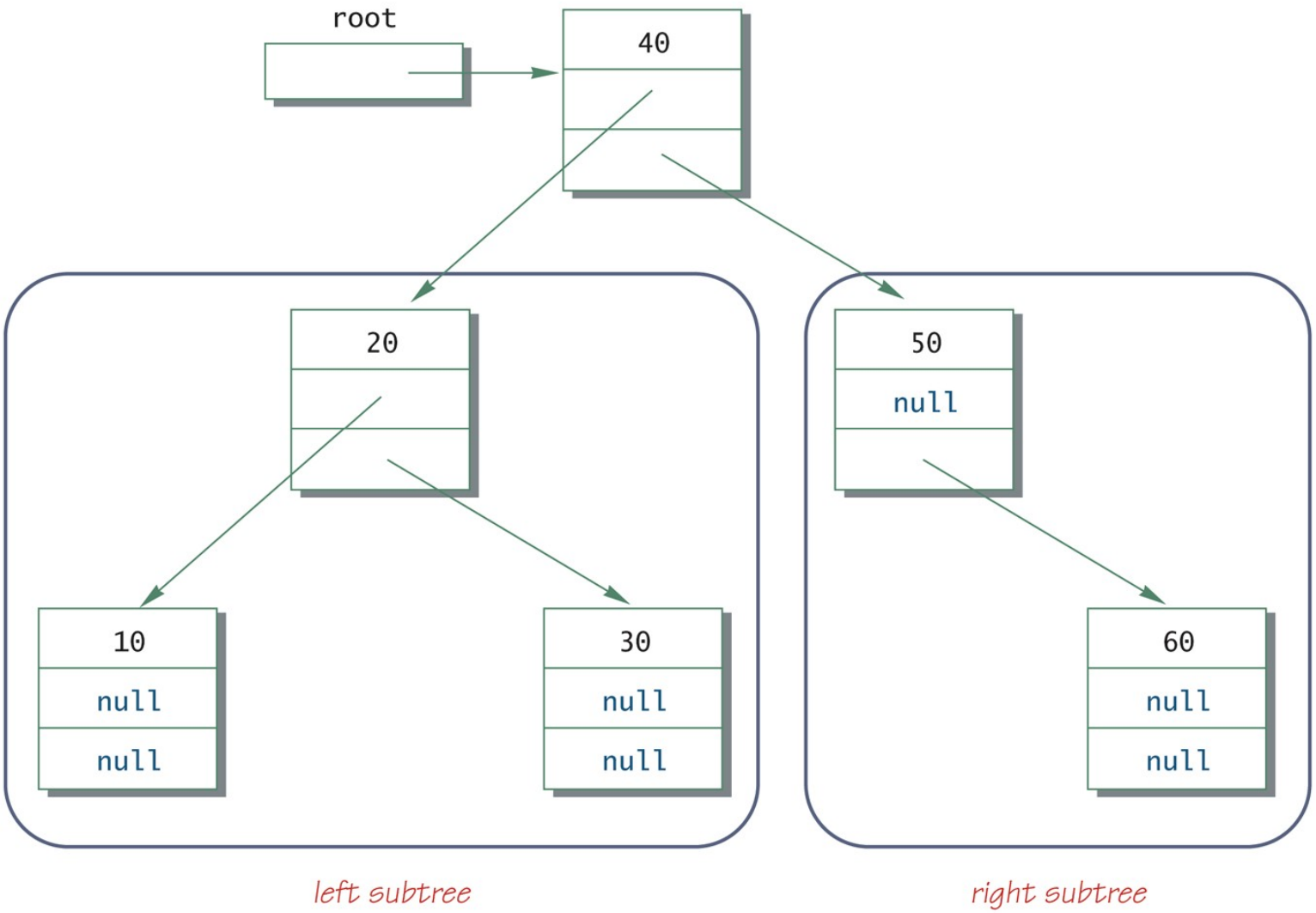
- Trees are a very important and widely used data structure
- Like linked lists, they are a structure based on nodes and links, but are more complicated than linked lists
  - All trees have a node called the *root*
  - Each node in a tree can be reached by following the links from the root to the node
  - There are no cycles in a tree: Following the links will always lead to an "end"

Trees

- A binary tree is the most common kind of tree
  - Each node in a binary tree has exactly two link instance variables
  - A binary tree must satisfy the Binary Search Tree Storage Rule
- The root of the tree serves a purpose similar to that of the instance variable **`head`** in a linked list
  - The node whose reference is in the **`root`** instance variable is called the *root node*
- The nodes at the "end" of the tree are called *leaf nodes*
  - Both of the link instance variables in a leaf node are **`null`**

A Binary Tree (Part 1 of 2)

Display 15.24 A Binary Tree



(continued)

A Binary Tree (Part 2 of 2)

Display 15.24 A Binary Tree

```
1   public class IntTree
2   {
3       public class IntTreeNode
4       {
5           private int data;
6           private IntTreeNode leftLink;
7           private IntTreeNode rightLink;
8       } //End of IntTreeNode inner class

9       private IntTreeNode root;
      <The methods and other inner classes are not shown.>

10  }
```

Binary Search Tree Storage Rule

1. All the values in the left subtree must be less than the value in the root node
2. All the values in the right subtree must be greater than or equal to the value in the root node
3. This rule is applied recursively to each of the two subtrees

(The base case for the recursion is an empty tree)

Tree Properties

- Note that a tree has a recursive structure
  - Each tree has two subtrees whose root nodes are the nodes pointed to by the **`leftLink`** and **`rightLink`** of the root node
  - This makes it possible to process trees using recursive algorithms
- If the values of a tree satisfying the Binary Search Tree Storage Rule are output using *Inorder Processing*, then the values will be output in order from smallest to largest

Preorder Processing

1. Process the data in the root node
2. Process the left subtree
3. Process the right subtree

Inorder Processing

1. Process the left subtree
2. Process the data in the root node
3. Process the right subtree

Postorder Processing

1. Process the left subtree
2. Process the right subtree
3. Process the data in the root node

A Binary Search Tree for Integers (Part 1 of 6)

Display 15.25 A Binary Search Tree for Integers

```
1  /**
2   Class invariant: The tree satisfies the binary search tree storage rule.
3  */
4  public class IntTree
5  {
6      private static class IntTreeNode
7      {
8          private int data;
9          private IntTreeNode leftLink;
10         private IntTreeNode rightLink;
11
```

The only reason this inner class is static is that it is used in the static methods `insertInSubtree`, `isInSubtree`, *and* `showElementsInSubtree`.

(continued)

A Binary Search Tree for Integers (Part 2 of 6)

Display 15.25 A Binary Search Tree for Integers

```
12          public IntTreeNode(int newData, IntTreeNode newLeftLink,
13                                          IntTreeNode newRightLink)
14          {
15                data = newData;
16                leftLink = newLeftLink;
17                rightLink = newRightLink;
18          }
19      } //End of IntTreeNode inner class


20      private IntTreeNode root;

21      public IntTree()
22      {
23          root = null;
24      }
```

This class should have more methods. This is just a sample of possible methods.

(continued)

A Binary Search Tree for Integers (Part 3 of 6)

Display 15.25 A Binary Search Tree for Integers

```
25      public void add(int item)
26      {
27          root = insertInSubtree(item, root);
28      }

29      public boolean contains(int item)
30      {
31          return isInSubtree(item, root);
32      }

33      public void showElements()
34      {
35          showElementsInSubtree(root);
36      }
```

(continued)

A Binary Search Tree for Integers (Part 4 of 6)

Display 15.25 A Binary Search Tree for Integers

```
37      /**
38       Returns the root node of a tree that is the tree with root node
39       subTreeRoot, but with a new node added that contains item.
40      */
41      private static IntTreeNode insertInSubtree(int item,
42                                                 IntTreeNode subTreeRoot)
43      {
44          if (subTreeRoot == null)
45              return new IntTreeNode(item, null, null);
46          else if (item < subTreeRoot.data)
47          {
48              subTreeRoot.leftLink = insertInSubtree(item, subTreeRoot.leftLink);
49              return subTreeRoot;
50          }
```

(continued)

A Binary Search Tree for Integers (Part 5 of 6)

Display 15.25 A Binary Search Tree for Integers

```
51          else //item >= subTreeRoot.data
52          {
53              subTreeRoot.rightLink = insertInSubtree(item, subTreeRoot.rightLink);
54              return subTreeRoot;
55          }
56      }

57      private static boolean isInSubtree(int item, IntTreeNode subTreeRoot)
58      {
59          if (subTreeRoot == null)
60              return false;
61          else if (subTreeRoot.data == item)
62              return true;
63          else if (item < subTreeRoot.data)
64              return isInSubtree(item, subTreeRoot.leftLink);
65          else //item >= link.data
66              return isInSubtree(item, subTreeRoot.rightLink);
67      }
```

(continued)

A Binary Search Tree for Integers (Part 6 of 6)

Display 15.25 A Binary Search Tree for Integers

```
68      private static void showElementsInSubtree(IntTreeNode subTreeRoot)
69      {//Uses inorder traversal.
70          if (subTreeRoot != null)
71          {
72              showElementsInSubtree(subTreeRoot.leftLink);
73              System.out.print(subTreeRoot.data + " ");
74              showElementsInSubtree(subTreeRoot.rightLink);
75          }//else do nothing. Empty tree has nothing to display.
76      }
77  }
```

Demonstration Program for the Binary Search Tree (Part 1 of 3)

Display 15.26 Demonstration Program for the Binary Search Tree

```
1  import java.util.Scanner;

2  public class BinarySearchTreeDemo
3  {
4     public static void main(String[] args)
5     {
6        Scanner keyboard = new Scanner(System.in);
7        IntTree tree = new IntTree();
```

(continued)

Demonstration Program for the Binary Search Tree (Part 2 of 3)

Display 15.26 Demonstration Program for the Binary Search Tree

```
 8          System.out.println("Enter a list of nonnegative integers.");
 9          System.out.println("Place a negative integer at the end.");
10          int next = keyboard.nextInt();
11          while (next >= 0)
12          {
13              tree.add(next);
14              next = keyboard.nextInt();
15          }

16          System.out.println("In sorted order:");
17          tree.showElements();
18      }
19  }
```

(continued)

Demonstration Program for the Binary Search Tree (Part 3 of 3)

Display 15.26 Demonstration Program for the Binary Search Tree

SAMPLE DIALOGUE

```
Enter a list of nonnegative integers.
Place a negative integer at the end.
40
30
20
10
11
22
33
44
-1
In sorted order:
10 11 20 22 30 33 40 44
```

Efficiency of Binary Search Trees

- A Binary search trees that is as short as possible can be processed most efficiently
  - A short tree is one where all paths from root to a leaf differ by at most one node
- When this is so, the search method `isInSubtree` is about as efficient as the binary search on a sorted array
  - Its worst-case running time is O(log *n*), where *n* is the number of nodes in the tree

Efficiency of Binary Search Trees

- As a tree becomes more tall and thin, this efficiency falls off
  - In the worst case, it is the same as that of searching a linked list with the same number of nodes
- Maintaining a tree so that it remains short and fat, as nodes are added, is known as *balancing the tree*
  - A tree maintained in this manner is called a *balanced tree*

Questions?